Airframe Aerodynamic Design

Braxton Lopez

18 March 2026

Introduction

The purpose of this optimization is to reduce the amount of induced drag on a wing while maintaining certain conditions. The following is an optimization of the chord distribution for a wing with an $8m$ span, 5° angle of attack, freestream velocity of $1m/s$, and the wing must be capable of carrying a 1.7N load. VortexLattice.jl is used to find the lift and drag, and SNOW.jl is used to optimize the chord distribution. Additional optimizations were also run, the details for which can be found in *Additional Optimizations*.

Methods

The purpose of this optimization is to reduce the amount of induced drag under certain conditions. Figure 1 represents the results of this optimization; however, Figure 2 represents a wing optimized for the highest efficiency ratio ($\frac{Lift}{InducedDrag}$). They are nearly identical; however, optimizing for efficiency provides a much smoother chord distribution. The constraints followed are:

- The wing span b must be $8m$
- The wing must be capable of carrying 1.7N of weight
- The angle of attack must be 5°
- The freestream velocity v must be $1m/s$
- No twist

A 'monotonicity' constraint was also added in order to decrease the jaggedness of the chord distribution. Without this constraint, the optimizer provided geometry that is impossible/extremely difficult to manufacture in real world applications. Using the Diff() function found in Listing 1 allows the solver to find the difference between each chord, and the constraints from Listing 2 force the wing's chord to decrease as the distance y from the root increases.

Listing 1: Diff() Function

```
chord_diffs = diff(x)
for i in 1:length(chord_diffs)
    g[1 + i] = chord_diffs[i]
end
```

Listing 2: Diff() Function Constraints

```
lg = vcat([1.7], fill(-Inf, nx - 1))
ug = vcat([Inf], fill(0.0, nx - 1))
```

The first conditions in Listing 2 are the bounds on lift and the subsequent ones are those for the differences between the chord lengths. The following optimization was run using 20

spanwise panels per half-wing and 4 chord-wise panels. The quarter chord of all sections are aligned using the following code:

Listing 3: Quarter-Chord Alignment

```
chord = x
xle = 0.25 * x[1] .- 0.25 .* chord
```

The initial conditions for each chord were 1m, and the upper and lower bounds for the optimizer were 1.6m and 0.1m, respectively.

VortexLattice.jl was used to find the lift, drag, and efficiency, which the optimizer SNOW.jl then used to optimize the chord distribution. The exact process is detailed in Appendix A. The following function was used to plot the results, and note that VortexLattice was only used to optimize a half-wing, with the results mirrored to depict a full wingspan.

Listing 4: Plotting Function

```
function plotting_the_wing(xopt)
    # Reconstruct geometry from xopt
    b_half = 4.0 # (bref * 0.5)
    yle = range(0, b_half; length=nx)
    chord_opt = xopt
    xle_opt = 0.25 * xopt[1] .- 0.25 .* chord_opt
    xte_opt = xle_opt .+ chord_opt # Trailing edge x-position
    p1 = plot(yle, xle_opt, label="Leading␣Edge", color=:blue, lw
       =2)
    plot!(yle, xte_opt, label="Trailing␣Edge", color=:red, lw=2,
       fillrange=xle_opt, fillalpha=0.2)
    plot!(title="Optimized␣Wing␣Planform", xlabel="Spanwise␣
       Position␣(y)", ylabel="Chordwise␣Position␣(x)", yflip=true)
    # Mirror to see the full wing
    plot!(-yle, xle_opt, label="", color=:blue, lw=2)
    plot!(-yle, xte_opt, label="", color=:red, lw=2, fillrange=
       xle_opt, fillalpha=0.2)
end
```

Results

Figure 1 represents the results of optimization 1 (optimizing induced drag); however, Figure 2 represents a wing optimized for the highest efficiency ratio ($\frac{Lift}{InducedDrag}$). They are nearly identical; however, optimizing for efficiency provides a much smoother chord distribution.

When solving for induced drag, the optimizer stated that it reached the maximum number of iterations and provided the chord distribution found in Figure 2. This is likely why the chord distribution is more jagged, as it did not finish optimizing the distribution

(though this is difficult to see in the figures below). In the case that the optimizer allowed for more iterations, it would likely smooth out and be nearly identical to the chord distribution when optimizing efficiency.

When solving for efficiency $\frac{Lift}{InducedDrag}$, the solver reported convergence to a point of local infeasibility, implying that it found the most accurate solution possible under the given conditions. In other words, if it continued to optimize the chord distribution, it would break the constraints. The results are shown in Figure 2, and although it is difficult to see, Figure 1 is more jagged than Figure 2.

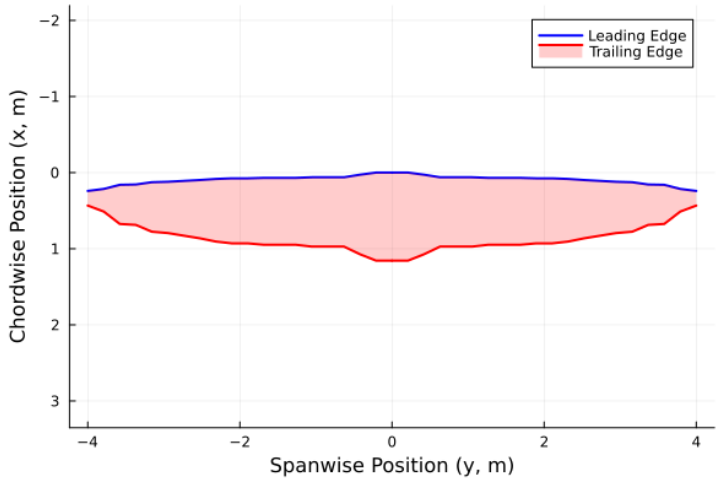

Figure 1: Optimal Chord Distribution for a wing with a span b of 8m, minimizing the induced drag.

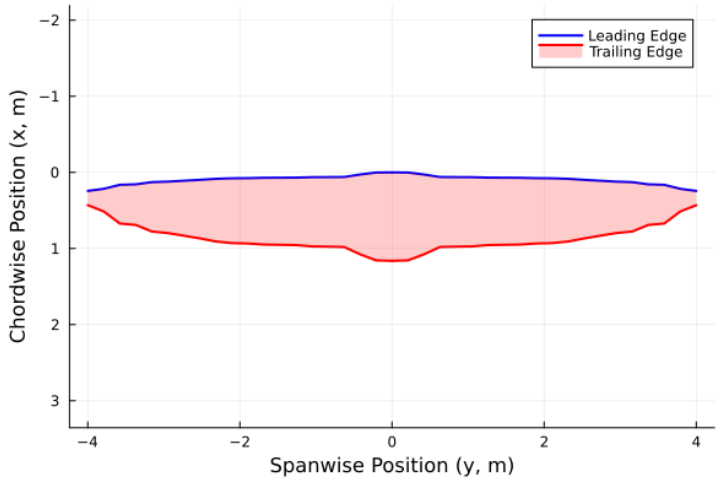

Figure 2: Optimal Chord Distribution for a wing with a span b of 8m, maximizing the ratio of lift to induced drag.

The Supermarine Spitfire, perhaps the most famous application of elliptical wing design, provides a useful benchmark for evaluating these results. While the optimized distribution is generally similar in shape, several simplifying assumptions prevent an exact match. This is due to the optimization's omission of several factors, as listed below. SNOW.jl optimized only the chord distribution, solving for minimal induced drag, yet there are several other factors that need to be included for a more accurate, idealized model.

The Supermarine Spitfire's wing had a trailing edge that was twisted, with the angle of incidence decreasing from $+2°$ at the root to $-0.5°$ at the tip, which allowed the wing to stall at the roots before the tips. The optimizations above were solved using a constant angle of attack $\alpha = 5°$, and is also unable to account for stall. The Spitfire has a dihedral angle of $6°$, whereas $\phi = 0°$ for the optimizations. The Spitfire also has a decreasing thickness-to-chord ratio, with 13% at the root, and 9% at the tip, whereas the optimization doesn't factor in the thickness of the wing, as VortexLattice.jl is unable to use this information. The optimization also doesn't include the stresses/bending moments that would be created, and a more accurate idealized model would factor in constraints caused by the material used in the wing.

This optimization was performed using 20 spanwise panels and 4 chordwise panels, leaving room for increased accuracy. SNOW.jl took approximately 30 seconds to optimize the efficiency and 530 seconds to minimize the induced drag. As I increased the number of spanwise panels to 200, it caused the optimizer to take over an hour to solve it.

In conclusion, there are many ways in which this optimization may be improved to more accurately represent the restrictions found for physical models. The optimization above suggests that in terms of optimizing efficiency and minimizing induced drag, an elliptical-type shape will perform best; however, there are constraints that must be obeyed when manufacturing these wings, originating from both material properties and factors not included in the optimization. For a breakdown of how varying the tolerance of the optimizer or its constraints affects a solution, see *Optimization 5* from the section *Additional Optimizations*, found below.

Additional Optimizations

Optimization 3-4 The following are the results of an optimization that includes the angle of attack α as a design variable, while including the previous design variables (found in the two optimizations detailed above). Otherwise, all constraints or initial conditions are the same as previously stated and can be found in Appendix A. The initial angle of attack was 5°, and the upper and lower bounds were 10° and 0°, respectively. Figure 1 depicts the results analyzing efficiency $\frac{Lift}{InducedDrag}$ with 20 spanwise panels, and Figure 2 depicts the results using 35 spanwise panels.

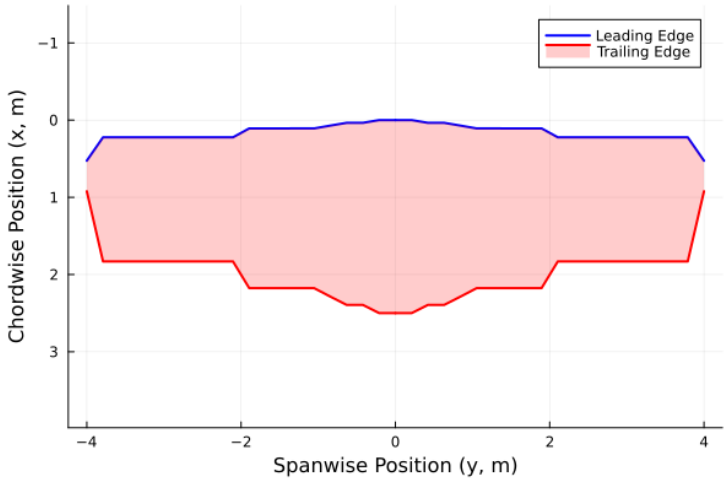

Figure 3: Optimal Chord Distribution for a wing with a span b of 8m, maximizing the ratio of lift to induced drag using 20 spanwise panels.

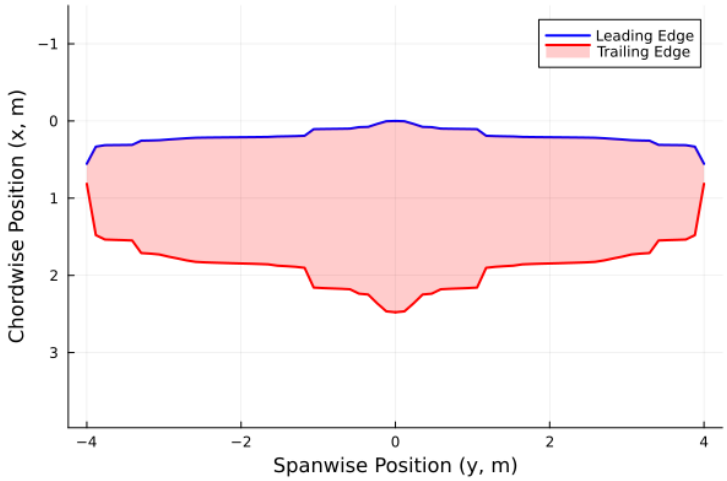

Figure 4: Optimal Chord Distribution for a wing with a span b of 8m, maximizing the ratio of lift to induced drag using 35 spanwise panels.

The optimized angle of attack that the optimizer returned was 3.2904°, and the Lift-to-Induced-Drag Ratio was 66.39398, which is lower than it initially was with a set angle of attack of 5°. This means that this design has a lower efficiency; however, it does have a larger total lift, increasing the load capacity from $1.7N$ to $4.87N$. This does, however, come with the disadvantage of having a root chord over twice as long as those in the previous optimizations (found in the section *Results*).

Optimization 5 Optimization 5 is nearly identical to optimization 3. It optimizes efficiency ($\frac{Lift}{InducedDrag}$) and follows the same constraints, with the exception of the maximum allowable chord length, which was adjusted from $2.5m$ to $1.5m$. It also uses 35 spanwise panels instead of 20.

Listing 5: Upper Bounds

```
ux = vcat(fill(1.5, nx), 10.0*pi/180)
```

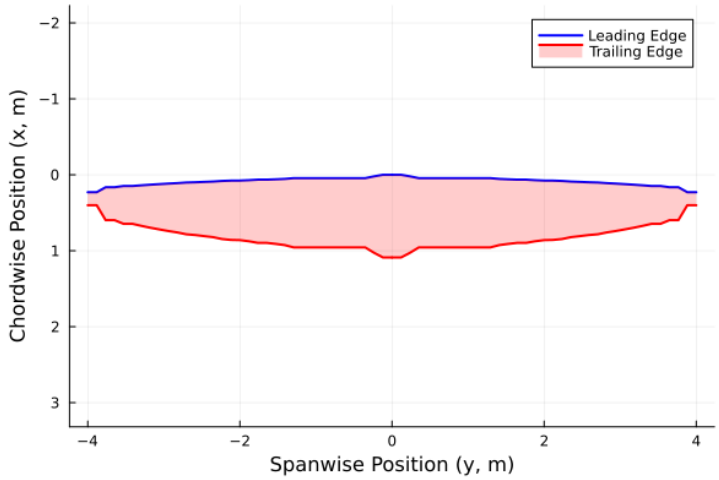

Figure 5: Optimal Chord Distribution for a wing with a span b of 8m, maximizing the ratio of lift to induced drag using 35 spanwise panels.

This provides a solution more similar to that found in optimizations 1-2. The Lift-to-Induced-Drag Ratio is 75.84 and the optimized angle of attack is 5.60°. This is an example of how varying the tolerance of the optimizer will affect the solution. Optimization 5 found a wing with a significant increase in efficiency (as compared to Optimization 4), increasing it from 66.39 to 75.84. It did decrease the load bearing capacity from 4.87N to 1.7N, yet it significantly reduced the amount of material required to produce the wing. For this reason, it's important to be aware of how the bounds affect the solution and to clarify for the solver what properties (i.e. most lift or highest efficiency) are most important. Proper constraints are also critical, and as an example, when the minimum lift produced is changed from 1.7N to 17N, the solver breaks and is unable to find any chord distribution that satisfies the given requirements.

Also note that for optimization 5, increasing the number of spanwise panels from 20 to 35 increases the time required to solve the optimization from 7 minutes to 64 minutes, and is much more computationally expensive. This effect is drastically increased as the number of panels is increased.

Appendix A - Code

Listing 6: Simulation Code

```
#FUNCTIONING OPTIMIZATIONS 1-2 WITH PARAMETRIC COMPLEX TYPES
using SNOW
using Plots
using VortexLattice
using Interpolations
using FLOWMath

struct WingParams{T}
    bref::T
    Vinf::T
    alpha::T
    density::T
    cref::T
    ns::Int
    nc::Int
end

function WingParams(bref, Vinf, alpha_deg, density, cref, ns, nc)
    T = promote_type(typeof(bref), typeof(Vinf), typeof(alpha_deg)
       , typeof(density))
    return WingParams{T}(T(bref), T(Vinf), T(alpha_deg * pi / 180)
       , T(density), T(cref), ns, nc)
end

function optimization(g, x)
    density = params.density
    bref = params.bref # reference span
    n = 20     # number of spanwise panels
       = range(0,   /2; length=n)
    b_half = bref*0.5
    yle = range(0, b_half; length=n)
    zle = zeros(n)
    chord = x
    xle = 0.25 * x[1] .- 0.25 .* chord
    theta = zeros(n)
    phi   = zeros(n)
    fc = fill((xc) -> 0.0, n)

    ns = params.ns # number of spanwise panels.
    nc = params.nc  # number of chordwise panels.
    spacing_s = Sine() # spanwise discretization scheme.
    spacing_c = Sine() # chordwise discretization scheme.
```

```
    grid, ratio = wing_to_grid(xle, yle, zle, chord, theta, phi,
       ns, nc;
    fc = fc, spacing_s=spacing_s, spacing_c=spacing_c, mirror=true
       )
    grids = [grid] #list of grids in the system (only one wing in
       this case).
    ratios = [ratio] #list of ratios in the system (only one wing
       in this case).
    system = System(grids; ratios) # Combines grids and ratios in
       a System object,

    y_half = collect(yle)          # 0     b/2
    c_half = chord                 # same length
    Sref = 2 * trapz(y_half, c_half)
    cref = params.cref  # reference chord
    rref = [0.50, 0.0, 0.0] # reference location for rotations/
       moments (typically the center of gravity)
    Vinf = params.Vinf # reference velocity (magnitude)
    ref = Reference(Sref, cref, bref, rref, Vinf) #Creates a
       Reference object that stores the reference parameters used
    #in the aerodynamic analysis.

    alpha = 5 * pi/180 # angle of attack
    beta = 0.0 # sideslip angle. The angle between the aircraft's
        forwardpointing  axis and the actual direction the air
    #is coming from. When you set beta = 0.0, you are telling the
       solver the aircraft has no sideways motion relative to
    #the airflow.
    Omega = [0.0, 0.0, 0.0] # rotational velocity around the
       reference location (rref). This vector represents the
       angular
    #velocity of the aircraft in three-dimensional space, with
       components along the x, y, and z axes. Setting it to
    #[0.0, 0.0, 0.0] indicates that the aircraft is not rotating.
    fs = Freestream(Vinf, alpha, beta, Omega) #Combines the
       freestream parameters into a Freestream object, which is
       used
    #to define the incoming airflow conditions for the aerodynamic
        analysis.

    symmetric = false #When set to false, the solver considers
       both sides of the aircraft separately, allowing for
       asymmetric
    #flow conditions and forces. This is important for accurately
       capturing the aerodynamic behavior of the aircraft,
```

```
    #especially during maneuvers (anytime theres a rolling and yawing moment) or in the presence of crosswinds. When set
    #to false, the solver does not assume symmetry about the aircraft's centerline and cancel out forces and moments.

    steady_analysis!(system, ref, fs; symmetric)
    CF, CM = body_forces(system; frame=Wind()) #This creates the force and moment vectors in the wind frame of reference.
    #Of the force vector, the x-component is the drag force, the y-component is the side force, and the z-component is
    #the lift force. Of the moment vector, the x-component is the rolling moment, the y-component is the pitching moment,
    #and the z-component is the yawing moment.
    # extract aerodynamic forces from the vectors created above.
    CD, CY, CL = CF
    Cl, Cm, Cn = CM
    CDiff = far_field_drag(system)#Computes the drag contribution from the far-field induced drag, which is the drag caused by the
    #vortices shed into the wake behind the lifting surfaces. This function calculates the far-field drag based on the vortex strengths
    #and wake geometry determined during the steady-state analysis. The result is stored in the variable "CDiff".
    #Simply put --> It calculated the coefficient of induced drag.
    # calculate lifting line coefficients
    cf, cm = lifting_line_coefficients(system; frame=Body()) #cf is the calculated section force coefficients along the
    #span, and cm is the calculated section moment coefficients along the span. They are distributions and not total forces.
    #frame=body means that its using the reference frame of the aircraft body (as opposed to wind frame used earlier).


    q = 0.5 * density * Vinf^2
    L_total = CL * q * Sref
    g[1] = L_total
    chord_diffs = diff(x)
    for i in 1:length(chord_diffs)
        g[1 + i] = chord_diffs[i]
    end
    D_induced = CDiff * q * Sref      # N
    LtD = -L_total / D_induced
    return LtD
```

```
end

const params = WingParams(8.0, 1.0, 5.0, 1.225, 1.0, 20, 4) #bref, Vinf, alpha, density, cref, ns, nc
nx = params.ns
x0 = fill(1.0, nx)
lx = fill(0.1, nx)
ux = fill(2.5, nx)
ng = 1 + (nx-1) # number of constraints
lg = vcat([1.7], fill(-Inf, nx - 1))
ug = vcat([Inf], fill(0.0, nx - 1))
options = Options(solver=IPOPT())  # choosing IPOPT solver

xopt, fopt, info = minimize(optimization, x0, ng, lx, ux, lg, ug, options)

println("xstar = ", xopt)
println("fstar = ", fopt)
println("info = ", info)


function plotting_the_wing(xopt)
    # Reconstruct geometry from xopt
    b_half = 4.0 # (bref * 0.5)
    yle = range(0, b_half; length=nx)
    chord_opt = xopt
    xle_opt = 0.25 * xopt[1] .- 0.25 .* chord_opt
    xte_opt = xle_opt .+ chord_opt # Trailing edge x-position
    p1 = plot(yle, xle_opt, label="Leading Edge", color=:blue, lw=2, aspect_ratio = 1)
    plot!(yle, xte_opt, label="Trailing Edge", color=:red, lw=2, fillrange=xle_opt, fillalpha=0.2)
    plot!(xlabel="Spanwise Position (y, m)", ylabel="Chordwise Position (x, m)", yflip=true)
    # Mirror to see the full wing
    plot!(-yle, xle_opt, label="", color=:blue, lw=2)
    plot!(-yle, xte_opt, label="", color=:red, lw=2, fillrange=xle_opt, fillalpha=0.2)
end
plotting_the_wing(xopt)
```